MPOO 2027-1 · GRUPO 06

Parcial 1 · correcciones

Versión D del examen del 7 de octubre. Para cada pregunta vienen el enunciado, tu respuesta tal como la enviaste en el portal y su corrección. La calificación no va aquí: la ves con el profesor.

Antes de empezar. Este archivo lo subió el profesor a tu rama. Trae el cambio a tu computadora antes de tu siguiente push, o tu push va a fallar:

```
git checkout entregas_hernandez_emilio
git pull --no-rebase
```

Qué tienes que hacer. En hojas, **a mano**, copia cada pregunta y escribe debajo tu respuesta corregida, usando la corrección que viene aquí. Donde la corrección ya te da la salida o el código, no basta con copiarlo: escribe **por qué** sale así, con la tabla de seguimiento de las variables, la causa del error o lo que devuelve tu método en cada renglón de la tabla. Debajo de esas preguntas viene qué escribir. Escanea las hojas o tómales foto, júntalas en un PDF y súbelo a esta misma carpeta, `entregas/hernandez_emilio/examen/p1/`, con tu push de siempre.

Pregunta 0.1 · Preguntas abiertas

Pregunta

Contesta con tus palabras, en una a tres oraciones. Puedes apoyarte en un ejemplo pequeño de código
¿Qué hace new al crear un objeto? Explica qué recibe la variable a la que asignas esa expresión.

Tu respuesta

```
La variable recibe todas las estancias y parametros que uno le agregue al objeto, y las
caracteristicas que tengan limite o una forma de realizarse muy especifica.
```

Corrección. La respuesta no dice qué hace new ni qué recibe la variable; habla de 'estancias y parámetros' sin una idea válida. Lo correcto: new crea la instancia y ejecuta el constructor para inicializarla, y la variable recibe una referencia a ese objeto.

Pregunta 0.2 · Preguntas abiertas

Pregunta

¿Qué significa void en la cabecera de un método? Explica si ese método puede modificar un atributo.

Tu respuesta

```
El void en la cabecera significa que no retornara nada en especifico, es por que es la
"plantilla" para cada metodo, el metodo si puede modificar el atributo ya que se encuentra
dentro de la funcion principal y se encarga de darle la estructura que requiere.
```

Corrección. Bien: void significa que el método no devuelve nada y que sí puede modificar un atributo. La justificación es incorrecta: void no tiene que ver con una 'plantilla' ni con estar dentro de la 'función principal'. Lo que ocurre es que el método actúa sobre el estado del objeto aunque no entregue un resultado con return.

Pregunta 0.3 · Preguntas abiertas

Pregunta

¿Para qué sirve this? Explica cómo distingue un atributo de un parámetro que tiene el mismo nombre.

Tu respuesta

```
El this sirve para poder darle los elementos que requiere al objeto a programar, por ejemplo en un programa de clase de expendedora, se usaba el this.papas, para adquirir ese parametro dentro del objeto, lo distingue el atributo ya que se define en el principio del programa y se manda a "llamar" tipo una funcion anidada en C
```

Corrección. Reconoce que this.atributo se usa dentro del objeto (ejemplo this.papas), pero no dice que this es una referencia al objeto actual. Tampoco explica bien la distinción: en this.x = x, la izquierda es el atributo del objeto y la derecha el parámetro con el mismo nombre. La comparación con funciones anidadas de C no aplica.

Pregunta 0.4 · Preguntas abiertas

Pregunta

Java siempre pasa por valor. ¿Qué recibe un método cuando le pasas un int y cuando le pasas un arreglo?

Tu respuesta

```
Cuando le pasas un Int el metodo sabe que esta recibiendo una variable entera, y que se esta definiendo algo en especifico, cuando le pasa un arreglo es como si tuviera varias cajas (tamaño del arreglo) y en cada una se encuenrra un elemento que justo con el int fue definida y puesta en el lugar que se decide (indice)
```

Corrección. No menciona copias ni referencias; describe qué es un int y qué es un arreglo, sin decir qué recibe el método. Lo correcto: con un int recibe una copia del número y con un arreglo una copia de la referencia, así que modificar elementos sí afecta al arreglo original, pero reasignar el parámetro no cambia la variable de quien llama.

Pregunta 0.5 · Preguntas abiertas

Pregunta

¿En qué se distinguen JDK y JRE? Explica cuál necesitas para compilar un programa y por qué.

Tu respuesta

```
Se distinguen en que uno emula las funciones de linux en un dispositivo Windows, y el otro es un compilador nativo de linux. Necesitamos para compilar el JDK ya que es una maquina virtual que corre todos los comandos de linux como por ejemplo mkdir, cd, cp, ls, etc... y e sto es indispensable para poder compilar (javac), ejecutar (java) y desplazarse mejor dentro del sistema.
```

Corrección. Conserva una idea válida: para compilar con javac se necesita el JDK. La distinción es incorrecta, porque confunde JDK y JRE con emular Linux en Windows y con comandos como mkdir o cd. Lo correcto: el JRE es el entorno de ejecución (la JVM y sus bibliotecas) y el JDK añade herramientas de desarrollo, como javac.

R1 · ¿Qué imprime?

Pregunta

Escribe la salida exacta, una línea por renglón. El fragmento está en un main válido.

```
Integer base = 5;
Integer copia = base;
int[] aumentos = {1, 2, 3};
for (int aumento : aumentos) {
    base = base + aumento;
    System.out.println(base / 2);
}
String etiqueta = "D" + copia + 1;
System.out.println(etiqueta);
System.out.println((double) (base / 2));
```

Tu respuesta

```
34
D51
3.5
```

Corrección. D51 es correcta. '34' parece juntar las dos primeras líneas (3 y 4) sin salto de línea y falta el 5. La última línea es 5.0, no 3.5: base termina en 11, la división entera 11/2 da 5 y el cast posterior lo vuelve 5.0. La salida exacta era 3 / 4 / 5 / D51 / 5.0.

Qué escribir a mano. La salida ya la tienes en la corrección. Escribe por qué sale así: una tabla de seguimiento con el valor de cada variable en cada vuelta y en cada println.

R2 a) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
if (true) { int voltaje = 12; }
System.out.println(voltaje);
```

Tu respuesta

Clasificación

```
Funciona
```

Causa o salida

```
12
```

Corrección. Clasificación y salida incorrectas. No compila: voltaje se declara dentro del bloque del if y no existe fuera de él, aunque la condición sea true.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 b) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
StringBuilder ruta = new StringBuilder("Q");
ruta.append(2 + 3);
System.out.println(ruta);
```

Tu respuesta

Clasificación

```
Compila pero truena
```

Causa o salida

```
compila una vez pero la segunda truena por el metodo
```

> **Corrección.** Clasificación y causa incorrectas: no hay ninguna falla. Funciona e imprime Q5, porque primero se evalúa 2 + 3 = 5 y luego append lo agrega al StringBuilder.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 c) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
int[] canales = null;
System.out.println(canales.length);
```

Tu respuesta

Clasificación

```
No compila
```

Causa o salida

```
No puedes llamar al tamaño de un arreglo si el arreglo esta vacio
```

> **Corrección.** La clasificación es incorrecta: sí compila, pero truena en ejecución con NullPointerException. La causa se acerca a medias, porque el problema no es que el arreglo esté vacío (un arreglo vacío tiene length 0) sino que canales es null y no apunta a ningún arreglo.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R3 · Modelado: base, reto y justificación

Pregunta

Cada termostato guarda etiqueta fija y temperatura. El rango común del edificio es de 16 a 28 grados. Constructor y ajustar(int temperatura) reciben valores válidos; ajustar asigna. Añade una consulta de temperatura.

Base. Declara al menos dos atributos de instancia con tipo, uno static y una constante final (¿también static?). Completa constructor y dos métodos: cambio y consulta.

Reto. ¿Cómo declaras los dos extremos del rango para que sean comunes y no cambien?

Justifica. Una línea por decisión: tipo, instancia/clase, final o mutable, constructor y métodos.

Cuerpos de 1 a 3 líneas, sin estructuras de control ni main. Hay más de un modelo correcto. Un elemento sin justificar vale la mitad.

Tu respuesta

Clase

```
class Termostato{

    void final Termostato{
        int temp(int[] = 16)
        int rango = 12


        this.temp = temp

        return Termostato;
    }
}
Static Consulta{
    this.tempf = tempf
    return tempf;

}
Static Regulador{
    this.rango = rango
    return rango;
}

pseudocodigo:
Se define el objeto class y dentro de este se desarrolla el static en el cual contendra todos los elementos que debe de tener el objeto, antes declarados con int para que el metodo los pueda lamar utilizando el this.parametro.
```

Justificaciones

```
Se señala un objeto que es el termostato y se crean sus parametros de temperatura, y se mandan a llamar cuando la estancia lo requiere, se incluyen diferentes formas de consultar la temperatura y de ver el rango de temperatura que es termostato tiene, tambien se incluye el parametro de cada uno de los static
```

Corrección. Solo hay intentos aislados: una temperatura int, this.temp = temp y un método de consulta que devuelve un valor. Faltan los atributos de instancia declarados en la clase (final String etiqueta, int temperatura), el constructor con parámetros y sin void ni return, ajustar(int temperatura), y el reto: static final int MINIMO = 16 y MAXIMO = 28. Las justificaciones son genéricas y no explican tipo, instancia o clase, ni final o mutable.

R4 · Código con repetición

Pregunta

Devuelve el promedio entero de los valores pares de pulsos usando for-each. Si no hay pares, devuelve 0. El arreglo puede estar vacío, no es null y admite negativos. La suma cabe en int. No modifiques el arreglo.

```
static int promedioPares(int[] pulsos)
```

Ejemplos (argumentos → retorno):

{3, 8, 2, 5} → 5 · {1, 3, 5} → 0 · {-4, 3, 6} → 1

Escribe el método completo. No imprimas ni leas del teclado.

Tu respuesta

```
static int promedioPares(int[] pulsos)
for(i=0; i<pulsos.length;i++){
    int suma = promedioPares[0]+ i;
    int prom = suma/pulsos.length;
    return prom;
    for(i=0; i<pulsos.length;i++){
    int impar= i%2;
    return null;
}
}
PSEUDOCODIGO:
se usa un for que recorra el arreglo desde i indice 0 hasta i  = tamaño del arreglo, despues
se define una variable que se encargue de sumar los elementos que esten dentro del indice de
i, se usa otra variable que sea la division para que se haga el promedio y se retorna esa
variable. Y para descartar los pulsos impares se crea un each para que descarte con una
condicion aquellos numeros impares del arreglo.
```

Corrección. Cabecera correcta. Usa un for clásico en lugar de for-each, y el return dentro del ciclo termina en la primera vuelta. Además suma promedioPares[0] + i en lugar de los valores, divide entre pulsos.length en vez de entre la cantidad de pares, no filtra con if (valor % 2 == 0), no devuelve 0 cuando no hay pares y termina con return null en un método int. El pseudocódigo describe parte de la idea (recorrer, sumar, descartar impares, dividir); faltaba contar los pares y devolver 0 si la cuenta es cero.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.

R5 · Código con selección

Pregunta

Clasifica la temperatura entera de una cámara de pruebas. Usa if / else-if / else y devuelve el texto de la tabla. Los límites están incluidos. No imprimas ni leas del teclado.

```
static String estadoTermico(int temperatura)
```

Regla o valor recibido	Texto devuelto
Menor que 0 o mayor que 120	"INVALIDA"
0 a 29	"FRIA"
30 a 59	"TEMPLADA"
60 a 89	"CALIENTE"
90 a 120	"MUY CALIENTE"

Tu respuesta

```
if(temperatura<=0){
    system.out.println("Temperatura invalida");
}
if(temperatura <=29){
    Fria
}else{temperatura <=59)
    Templada
}else{temperatura <=89)
   caliente
if(temperatura<=120)
    Muy Caliente


_____________________________
CON SWITCH-CASE :ccc
for(temperatura > 0){
    system.out.println("Invalida")
}
switch(temperatura > 0){
    case 1: (temperatura < 29)
            return Fria
    case 2: (temperatura < 59)
            return Templada
    case 3: (temperatura < 89)
            return Caliente
    case 4: (temperatura < 120)
            return Muy Caliente

}
```

Corrección. Bien: las cuatro categorías válidas y las transiciones 29/30, 59/60 y 89/90 están en el orden correcto. Fallas: falta la firma static String estadoTermico(int temperatura); usa <= 0 (así 0 queda inválido aunque es FRIA) y no rechaza los mayores que 120; imprime en vez de devolver; y no hay return de los textos entre comillas ("FRIA", "MUY CALIENTE", etc.). Lo correcto era if (t < 0 || t > 120) return "INVALIDA"; else if (t < 30) return "FRIA"; y así hasta else return "MUY CALIENTE".

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.